



OBJECT ORIENTED PROGRAMMING OOP ASSIGNMENT 1

Smart Allocation Sytem

Submitted to: Dr. Muhammad Shahid Bhatti

Submitted by: AMNA KASHIF

SMART ALLOCATION SYSTEM

Problem scenario

COMSATS Lahore plans to introduce a Smart Campus Learning Space Allocation System for classrooms, laboratories, seminar rooms, and project spaces. Students, faculty members, administrators, security staff, maintenance staff, and disability support staff will use or depend on the system. The university has not prescribed one design. You must analyze the problem and propose an object oriented conceptual model. No Java implementation is required.

Task 1 - Stakeholder and requirement analysis:

Stakeholder identification:

- **Student:** needs to request space for either individual or group study, group projects, or student events requiring fair and transparent booking access.
- **Teacher:** needs high priority in booking for scheduled lectures, makeup classes or urgent educational meetings without the risk of last minute clashes.
- **Maintenance Staff:** needs the authority to inspect physical rooms and block any unsafe or damaged spaces from being reserved.
- **Disability Support Staff:** needs to ensure accessible rooms remain available for students with specific needs, while protecting the students personal disability information from being publically disclosed.

Three requirement conflicts and assumption:

1. Fair access to students vs high priority for faculty:

- **Conflict:** Faculty members require guaranteed spaces for academic meetings and makeup classes, which can lead to over reserving or blocking the students from booking space for group study.
- **Assumption:** Faculty members can book a class up to 30 days before the actual date on the other hand students can book only 7 days in advance with a limit of 2 rooms per student.

2. Administration active booking vs Maintenance Safety measures:

- **Conflict:** Maintenance staff must immediately block the unsafe rooms but these rooms might be already booked by the administration or faculty.

- **Assumption:** The maintenance staff must cancel all the existing confirmed bookings immediately upon flagging an unsafe condition. The system must relocate the affected users to a comparable alternative room or trigger an emergency notification.

3. Accessibility Priority vs. Student Fairness:

- **Conflict:** The disabled students need priority in booking accessible rooms but non disabled students need equal access to available rooms. At the same time the system cannot show why a specific room is prioritized or reserved without exposing the sensitive medical or disability information.
- **Assumption:** Accessible rooms are placed on a standard hold for general users until 48 hours before a time slot during which only students with registered accessibility profiles can book them. In order to protect privacy the system simply lists the room status as "Reserved" or "Held" without displaying disability details.

Task 2 - Object oriented conceptual model

Proposed Class:

Class 1 : User

- **Responsibility:** Represents any individual in the university system who has a personal identity and can be accessed through a register.
- **Attributes:** String userId , String fullName, String emailId.
- **Operations:** Boolean accessibilitystatus() , void updateprofile().

Class 2: Role

- **Responsibility:** Represents the changes in permissions, priority and advance booking limits that a user may have.
- **Attributes:** string roleName , int advanceBookingDaysLimit , int priorityLevel.
- **Operations:** boolean canBookinAdvance () , int priorityLevel().

Class 3: Room

- **Responsibility:** Represents and monitors the physical condition and accessibility of a room on campus.
- **Attributes:** string roomId , boolean isAccessible , int space.
- **Operations:** boolean isAvailable() , void setOperational status()

Class 4:Room Equipment

- **Responsibility:** Represents particular physical or technological hardware that are connected or installed in a room.
- **Attributes:** string name , string equipmentId.
- **Operations:** verify condition() , void equipmentDetails().

Class 5: Reservation

- **Responsibility:** Records the life cycle, time limits and the approval status for one room booking.
- **Attributes:** string reservationId , string status , int startTime , int endTime
- **Operations:** confirmReservation () , cancel Reservation().

Class 6: Disability service

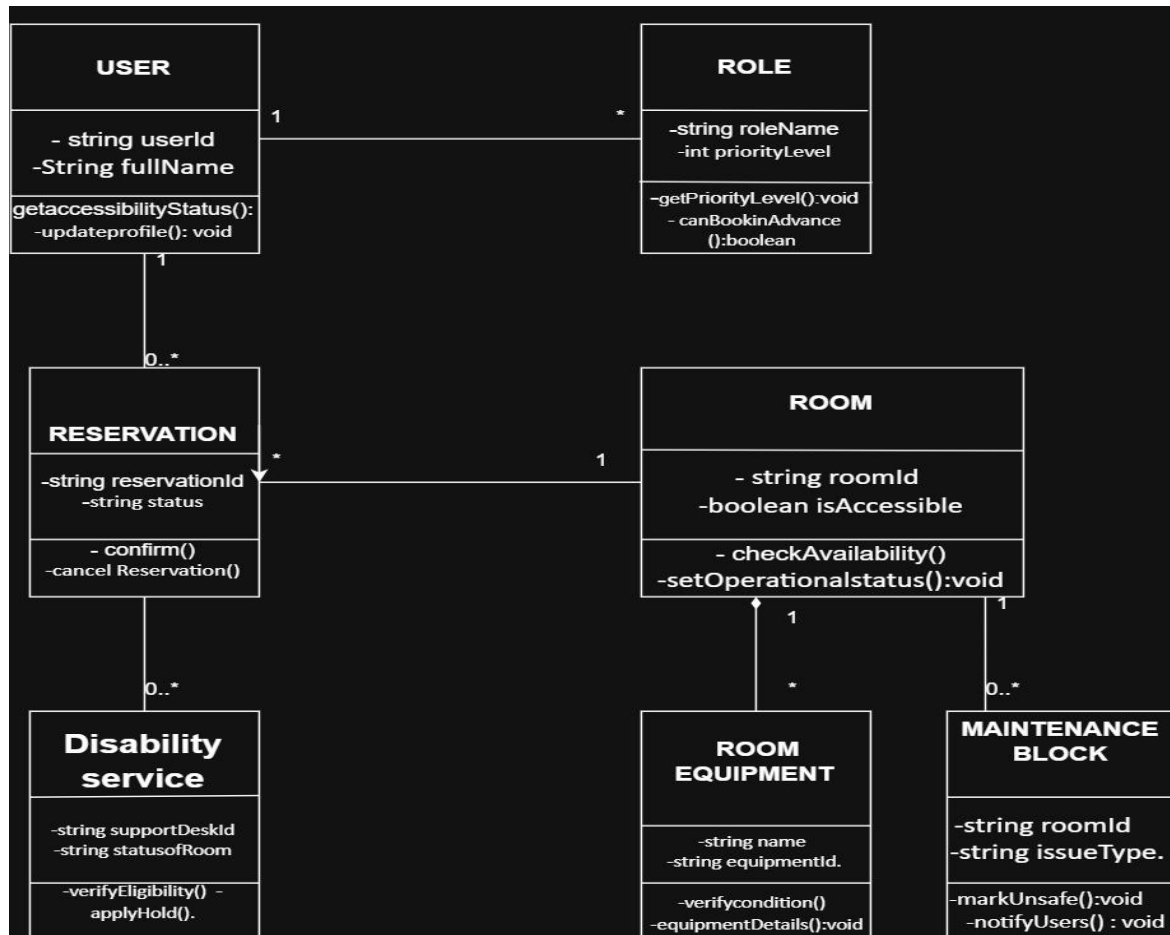
- **Responsibility:** Registers students for accessible rooms, checks special accommodation requests, and applies holds on accessible rooms.
- **Attributes:** string contactEmail, string supportDeskId , void statusofRoom.
- **Operations:** grantAccess() , verifyEligibility() , applyHold().

Class 7 : Maintenance block

- **Responsibility:** Indicates temporary closures in physical areas / rooms for safety or repair.
- **Attributes:** boolean blocked , string roomId , string issueType.
- **Operations:** void markUnsafe() , void notifyUsers.

UML Diagram:

Took help from Gemini in understanding and making UML diagram.



Task 3 Abstraction, encapsulation and invariants

- **CLASS “USER”**: Abstracts an on-campus physical person (such as a student, lecturer, staff member) into core software information, such as identity (userId), full name, and accessibility registration status (accessibilityStatus), and removes extraneous details from the real world such as age, hobbies, and academic transcriptions.
- **Class “ROOM”**: Abstracts a physical room/building space into core operational attributes required for scheduling, such as room ID, capacity, and whether it has accessible features (isAccessible), filtering out structural details like paint color, door dimensions, or wall materials.

- **Class "RESERVATION"**: Abstracts a real-world booking agreement into a temporal object with a unique booking ID, a status string and time bounds without considering subjective meeting agendas or conversation notes.

Used AI in understanding these concepts and answering.

Hiding information and controlled operations:

Class "USER":

- **Hidden Data:** accessibilityStatus (boolean) and userId (private variables).
- **Controlled Access:** Modified via updateProfile() and inspected via getAccessibilityStatus(), ensuring external classes cannot alter accessibility flags directly without validation.

Class "ROOM":

- **Hidden Data:** isAccessible and physical status flags.
- **Controlled Access:** Updated via setOperationalStatus() and queried via checkAvailability(), keeping room states synchronized.

Class "Reservation":

- **Hidden Data:** status string.
- **Controlled Access:** Changed strictly through state methods like confirm() and cancelReservation(), preventing unauthorized state manipulation.

Four Invariants:

- **Invariant 1** (Accessibility Protection Invariant): If a room is marked with isAccessible = true, it is not possible to assign a user to the room during the active Disability service priority hold window.
- **Invariant 2** If there is an active MAINTENANCE BLOCK (markUnsafe() is active) then the operational status of a ROOM is unavailable, which means that it is not possible to make a new Reservation for that ROOM.
- **Invariant 3** A ROOM EQUIPMENT instance must be parented to a single parent ROOM (enforced by composition).

- **Invariant 4** (Non-Overlapping Time Invariant): No two RESERVATION objects with state "CONFIRMED" can share the roomId and overlapping time slots.

The purpose of Encapsulation is to prevent the direct mutation of variables outside the class by using the validated operations. If fields were public, external code could bypass safety checks (forcing a room to confirm a booking while it's being maintained), which could result in invalid system states.

Task 4 Alternative designs and decision

This activity consists of comparing the User Role Representations.

We consider two architecture options for representing users that have multiple roles or changing roles (such as a student who switches to a teaching assistant role):

- **Option A** (Inheritance / Class Hierarchy): Sub classing User into Student, Faculty and Teaching Assistant.
- **Option B** (Composition / Role Objects - Recommended): A User object contains a set of detached Role objects (1 to*) with dynamic permissions.

3. Justified Decision

Selection: Option B (Composition / Role Objects).

Justification: This is explicitly stated in the prompt, and campus members often have overlapping roles (e.g., student as a TA), and permissions change throughout the semester. Inheritance is forced classification, while composition is dynamic reference of the role without changing the identity of the user.

Task 5 Behaviour and edge cases

There are five lifecycle states of a RESERVATION:

- **REQUESTED:** First submission by USER.
- **CONFIRMED:** The checkAvailability succeeded. Validated with disability priority holds.
- **REJECTED:** Unsuccessful because of double booking or violation of rules.

- **CANCELLED:** Cancelled by User or because of a MAINTENANCE BLOCK.
- **RELOCATED:** Moved to a more accessible room in the event of emergency closure.

Edge Case Handling

Two Simultaneous Requests for the Space:

Response: Managed by using thread safe transaction locks on ROOM object. The first request to acquire the lock becomes CONFIRMED. If the second simultaneous request passes the check, it goes into the REJECTED state, and asks the user to pick another window.

Responsible Object: Room (via checkAvailability()) control the state of the room, RESERVATION control the transition of the room's state.

After confirming, the students would go to Maintenance Block to get ready for the day's activities.

Response: If maintenance triggers call markUnsafe(), the system will ask the reservations that are affected. They will change from "Confirmed" to "Cancelled" (or auto-relocated). Notifyusers(): Sends alert in real-time.

Responsible Object: Maintenance block is part of which is responsible for locking and delegating the notification of the user to its handler.

3. Emergency Cancellation / Relocation:

Response: In response to safety hazards. People who make reservations in Disability are automatically re-routed to accessible equipment that is reserved.

Disability service works with ROOM to ensure accessibility compliance.

Task 6 Independent exploration

Role-Based Access Control

Concept Definition & Model Application

RBC is an authorization mechanism that determines who can access the system based on what job functions they have within the organization, and not on their name or identity. Individual rights are given to each student or instructor, or permissions are granted to

specific roles (e.g. StudentRole, DisabilitySupportRole, FacultyRole).Used AI to understand RBC.

In our model it directly enhances the interaction of user and ROLE Disability service. In addition, by assigning permissions to the role object, a user who has the Disability Support Role automatically can override booking holds on rooms. This separates permission logic, personal accounts safeguard disability privacy flags, and it maintains clean and scalable